Guidebook for the Lecture NLP with Deep Learning

TrustHLT

Prof. Dr. Ivan Habernal, Yassine Thlija

Winter Semester 2026/27

Guidebook to Exercise Sheet 2

This guide summarizes the concepts needed for Exercise Sheet 2. It combines the tensor fundamentals used in PyTorch with the differentiation concepts needed to understand gradient-based learning and neural-network computations.

1. Tensors: The Fundamental PyTorch Data Structure

A tensor is a generalization of vectors and matrices to an arbitrary number of dimensions.

- A 0-D tensor is a scalar (`torch.tensor(5)`).
- A 1-D tensor is a vector (`torch.tensor([1,2,3])`).
- A 2-D tensor is a matrix.
- A 3-D tensor or higher is used for structured data, such as images and batches.

Tensors hold numerical data and are the foundation of all computations in PyTorch, from deep learning to numerical simulation.

2. Creating Tensors

Use `torch.tensor()` to create tensors directly from Python lists:

```
tensor = torch.tensor([1, 2, 3, 4, 5, 6])
```

Shape and Size

Each tensor has a shape, which defines its dimensionality:

```
tensor.shape  # torch.Size([6])
```

You can reshape tensors using:

```
torch.reshape(tensor, (rows, cols))
```

Example transformations:

- $(2, 3)$: 2 rows, 3 columns,
- $(3, 2)$: 3 rows, 2 columns,
- $(3, 2, 1)$: a 3-D tensor with 3 “blocks”.

3. Tensor Initialization

PyTorch provides convenience functions for creating constant or sequential tensors.

Function	Description	Example
`torch.full(size, value)`	Tensor filled with a constant	`torch.full([2,3], 9)`
`torch.ones(size)`	Tensor of ones	`torch.ones([3,3])`
`torch.zeros(size)`	Tensor of zeros	`torch.zeros([3,3])`
`torch.arange(start, end)`	Sequence of numbers	`torch.arange(0,5)`

4. Stacking and Concatenation

`torch.stack()` joins multiple tensors along a new dimension.

```
x = torch.tensor([1, 4])
y = torch.tensor([2, 5])
z = torch.tensor([3, 6])

torch.stack([x, y, z])          # shape: (3, 2)
torch.stack([x, y, z], dim=1)  # shape: (2, 3)
```

- `dim=0` adds a new row dimension.
- `dim=1` stacks along columns.

5. Indexing and Slicing

You can access elements or slices of tensors using Python indexing syntax.

Operation	Description	Output
`tensor[2:-2]`	Elements between indices 2 and -2	`[3, 4]`
`tensor[::2]`	Every other element	e.g., `[1, 3, 5]`
`tensor[torch.tensor(0)]`	First row	`[1, 2, 3]`
`tensor[:, torch.tensor(0)]`	First column	`[1, 4, 7]`

Equivalent forms include:

```
tensor[0]
tensor[0, :]
tensor[0, ...]
```

6. Broadcasting

Broadcasting allows operations between tensors of different shapes by expanding one to match the other without copying data.

Expression	Description	Output shape
`a + b` (both scalars)	Simple addition	scalar
`a=[1,2], b=4`	`b` is broadcast to match `a`	`[5,6]`
`a=[[1,2,3],[4,5,6]], b=[10,20,30]`	`b` broadcast across rows	$(2, 3)$
`a=[[1,2,3],[4,5,6]], b=[[1],[2]]`	`b` broadcast across columns	$(2, 3)$

Broadcasting Rules

1. Start comparing shapes from the rightmost dimension.
2. Dimensions must be equal, or one must be 1.
3. Missing dimensions are treated as 1.

7. Arithmetic and Matrix Operations

PyTorch supports standard arithmetic operators.

Operation	Description
`+`, `-`, `*`, `/`, `%`, `//`	Element-wise operations
`@` or `torch.matmul()`	Matrix multiplication

Example:

```
a = torch.tensor([[1, 2]])
b = torch.tensor([[3], [4]])
a @ b  # matrix multiplication -> tensor([[11]])
```

8. Reduction Operations

Reduction operations collapse tensor dimensions by applying an aggregation function.

Function	Description	Example
`torch.max(t)`	Maximum element overall	`tensor(6)`
`torch.max(t, dim=0)`	Max along the first axis	values + indices
`torch.mean(t, dim=1)`	Mean along the second axis	`[0.33, 4.00]`
`torch.sum(x, dim=1, keepdim=True)`	Keeps reduced dimension	`[[3],[3]]`

9. Norms and Vector Lengths

The L2 norm, or Euclidean length, of a vector measures its distance from the origin.

```
torch.linalg.vector_norm(x, ord=2, dim=1)
```

- With `dim`: computes a per-row norm.
- Without `dim`: reduces the entire tensor.

10. Type Inference and Dtypes

Tensors can have different data types (`dtype`), such as:

- `torch.float32`, `torch.float64` for floating point,
- `torch.int32`, `torch.int64` for integers.

Be cautious: if you create a tensor from integers, operations such as `torch.mean()` may produce integer outputs unless you specify `dtype=torch.float`.

Example:

```
torch.mean(torch.tensor([1, 0, 0], dtype=torch.float))
```

11. Basics of Differentiation

Definition

Let $f : \mathbb{R} \to \mathbb{R}$ be a differentiable function. Its derivative with respect to x is defined as

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}.$$

The derivative measures how the function changes with respect to small changes in its input.

Common Derivatives

Function	Derivative
c	0
x	1
x^n	nx^{n-1}
e^x	e^x
$\ln(x)$	$\frac{1}{x}$
$\sin(x)$	$\cos(x)$
$\cos(x)$	$-\sin(x)$
$\sigma(x) = \frac{1}{1+e^{-x}}$	$\sigma(x)(1-\sigma(x))$
$\text{ReLU}(x) = \max(0, x)$	$\begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}$

12. Partial Derivatives

When a function depends on several variables, $f(x, y, z, \ldots)$, we can take the derivative with respect to one variable while keeping the others constant.

$$\frac{\partial f}{\partial x} = \lim_{h \to 0} \frac{f(x+h, y, z, \ldots) - f(x, y, z, \ldots)}{h}.$$

Example

For

$$f(x, y) = 3x^2 y + 2y^3,$$

the partial derivatives are

$$\frac{\partial f}{\partial x} = 6xy, \qquad \frac{\partial f}{\partial y} = 3x^2 + 6y^2.$$

13. Chain Rule

In deep learning, most functions are compositions of simpler functions. The chain rule allows us to compute the derivative of such compositions.

Single-variable Chain Rule

If $y = f(g(x))$, then

$$\frac{dy}{dx} = f'(g(x)) \cdot g'(x).$$

Multivariable Chain Rule

If $z = f(x, y)$ and both x and y depend on t, then

$$\frac{dz}{dt} = \frac{\partial f}{\partial x}\frac{dx}{dt} + \frac{\partial f}{\partial y}\frac{dy}{dt}.$$

This rule generalizes naturally to vector-valued and matrix-valued functions, which are essential for understanding gradient propagation in neural networks.

14. Derivatives of Some Deep Learning Functions

Sigmoid Function

The sigmoid function is defined as

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Its derivative is

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)).$$

This derivative is widely used in logistic regression and as an activation function in neural networks.

ReLU Function

The Rectified Linear Unit (ReLU) is defined as

$$\mathrm{ReLU}(x) = \max(0, x).$$

Its derivative is piecewise constant:

$$\mathrm{ReLU}'(x) = \begin{cases} 1, & x > 0, \\ 0, & x \leq 0. \end{cases}$$

Softmax Function

For a vector $z = (z_1, \ldots, z_K)$, the softmax is

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}.$$

The derivative of softmax with respect to z_j is

$$\frac{\partial \sigma(z_i)}{\partial z_j} = \sigma(z_i)\left(\delta_{ij} - \sigma(z_j)\right),$$

where δ_{ij} is the Kronecker delta.

Cross-Entropy Loss

For a one-hot encoded target vector y and predicted probabilities $\hat{y}$, the cross-entropy loss is

$$L = -\sum_{i=1}^{K} y_i \log(\hat{y}_i).$$

Its derivative with respect to the logits z_i, the inputs to the softmax, is

$$\frac{\partial L}{\partial z_i} = \hat{y}_i - y_i.$$